

Twelve questions to answer before you make your pick

1. Are you going to build your own RTOS, choose an open source option, or a proprietary one?
2. What is the total cost of ownership of the RTOS, including licence fees and royalties, if any, and the service or support charges? Is this affordable?
3. What is the memory footprint of the RTOS? Is it small enough?
4. Does the RTOS have all the features required by your application? On the other hand, is it too feature-rich, large and complicated? Is it possible to pick just what you want, and customise, if required?
5. Does it support the architecture you are working on? Multi-core, 64-bit support, etc, if required?
6. How good is the performance: code size and reliability, interrupt latency, context switch times, service overhead, etc?
7. Hope it does not guzzle too much power, memory or other resources?
8. Are there communications stacks (TCP/IP, USB), middleware, and other software available (flash file systems, graphics, etc)?
9. Is it easy to work on, without too sharp a learning curve?
10. Are there sufficient development tools? Are these easy to learn? Are development boards available?
11. Is it scalable and portable? Will it work well with other processors?
12. Does the vendor provide good support?

Although SHR is being made and tested for the ARM-based OpenMoko phones, the open source nature means that it can also be adapted for other architectures. In fact, the SHR team points out that many have shown interest in coming up with a version for the Palm Pre. It can be adapted for usage with personal digital assistants, navigation devices and Internet tablets.

Red Hat also has an RTOS based on the Linux kernel, which gives the user the ability to configure only what they find is necessary. It means that they may trim down the final binary to something that may fit their needs. Plus, the RTOS also inherits the benefits of the Linux parentage. "Any new feature of the Linux kernel becomes a new feature of our RTOS. Currently, we've included several tracing and performance utilities that will help out our customers in fine-tuning their system. We've also included more robust priority inheritance, mutual exclusion, application user interfaces....," says a Red Hat spokesperson.

Although Red Hat is focusing primarily on x86 architectures, the RTOS is available for ARM, PowerPC, SuperH, MIPS and more. Besides, it has been found suitable for a wide range of

projects, from the Garmin navigation system to large-scale database centres and embedded systems.

Multi-core RTOS

"The recent trends that we have observed in the embedded RTOS space are around the multi-core and hypervisor technology. Another positive trend is around 64-bit applications in embedded," says Venkatesh Kumaran, country manager-India, Wind River Systems.

Speaking of multi-core processors, the first question that arises in your mind is—cannot any RTOS be used for embedded systems using multi-core processors? You need an RTOS capable of shared memory communication, integrated on-chip I/O support, intelligent cache utilisation strategies, inter-core interrupts, multithreading and multiprocessing—that is, seamless system calls across all processors, symmetric multiprocessing, optional asymmetric multiprocessing, etc.

Keeping this popularity around multi-core in mind, Wind River has initiated a number of solutions targeting this space. They have on-chip debugging solutions that can debug up to eight cores. And, with their hyper-

visor, you can seamlessly move legacy applications to newer platforms.

The hypervisor technology, at the basic level, can be seen as another name for virtualisation. It is a 'virtual machine monitor' that can be implemented at either or both the hardware and software levels to enable multiple operating systems to run concurrently on a host computer. Virtualisation is essential to ensure full CPU utilisation in a multi-core system.

The technology has recently been extended to embedded systems as well, to enable a high-level operating system interface for application programming, such as Linux or Microsoft Windows. And, at the same time, maintaining traditional real-time operating system application programming interfaces (APIs).

In accordance with the definition of real-time operating systems, embedded hypervisors should also be capable of real-time operation, be it small in size, or using less resources. Hypervisors can support only embedded architectures that allow memory protection, as it is the key requirement for virtualisation. Wind River, OK-labs, VMware, eSOL, etc, have hypervisor solutions for the embedded space.

64-bit RTOS

Regular appliances do not require high-end systems, and 8-bit and 16-bit frameworks are more than sufficient for them. However, there are more complex embedded systems that require greater degrees of computing and much faster processing. For such requirements, the embedded RTOS space is now opting for 64-bit platforms. 64-bit microcontroller units and 64-bit floating point units attain a level of precision that is unimaginable with 32-bit systems. This makes these high-end processors suitable for critical telecommunication systems, process control, and more.

"Today, there are large amounts of data stored in the memory for higher performance. We usually believe that only a 64-bit operating system can suit